WP3 · Turbine parts in private_meshes (student)

A student turns every single GLB of the wind turbine model into an independently loadable object in `sunava/private_meshes`, so each part shows in the web catalogue, loads in CRAM and can be viewed in CRAMERA.

Goal and result

Today the turbine's 102 GLB meshes exist only inside the turbine model, positioned for the assembled turbine. Afterwards each part is a self-contained object in the shared AICOR asset library, the same way the 164 furniture and laboratory objects already are:

- one folder per part with its GLB, a `model.urdf`, rendered previews, a README and licence notices;
- a new turbine category in the web catalogue (https://sunava.github.io/private_meshes/), where every part can be browsed and downloaded;
- every part loads in CRAM with `URDFParser.from_file(...)` and shows in CRAMERA;
- the folders are produced by a script from the turbine model, not edited by hand, so a new export of the turbine updates them.

The assembled turbine as an environment (like the medical laboratory) is a stretch goal.

Repositories

Repository	Role in this work package	Access
`sunava/private_meshes`	Target. `objects/<category>/<id>-<slug>/` folders, per-category `collection.json` for the catalogue, `scripts/validate_resources.py`, `scripts/build_site.py`, Blender render scripts (`scripts/render_laboratory.py`), tests, GitHub Pages workflow. BSD-3-Clause; maintained by Vanessa Hassouna	Public; contribute through a fork and a pull request

Repository	Role in this work package	Access
`sorinar329/cognitive_robot_abstract_machine`, branch `windturbine_maintenance`	Source of the 102 GLBs (`models/<group>/*.glb`, textures embedded) and their OBJ/MTL versions; the turbine generator (`turbine/`: parts, frames, IEA dimensions, URDF export); scripts such as `build_gallery.py`, `build_cramera_bundle.py`, `build_ontology.py`; the turbine ontology (`ontology/windturbine.owl`, `.ttl`)	Public fork; clone sparsely (`models/`, `turbine/`, `scripts/`, `ontology/`)
`windturbine_model`	The Blender model the turbine description is generated from	Not public; access to be arranged
`sorinar329/windturbine_description`	Assembled turbine as a ROS URDF package: link and joint names, frames, materials, fault scenarios with ground truth. Reference for names and for where each part sits	Public
`cram2/cognitive_robot_abstract_machine`	`semantic_digital_twin`'s `URDFParser` to check every object loads in CRAM. The turbine's semantic annotations (`Tower`, `Nacelle`, `Hub`, `RotorBlade`, `Anemometer`) are on branch `wind-and-owl` of the fork above, for a `semantics.json` per object	Public
`AICOR-Solutions/cramera`	Viewer for the visual check in CRAM's 3D view	Public
`IEAWindSystems/IEA-3.4-130-RWT`	Origin of the turbine's dimensions (Apache-2.0); cited in every notice	Public, read only

Tools: Python 3.12, Blender 5.2 or newer (the render scripts need native WebP output), a CRAM environment for the load check.

Starting material

102 GLB files in nine groups on branch `windturbine_maintenance` (22 MB). Many parts come in pairs, a healthy and a faulty version of the same part (for example `brake_disc_ok` and `brake_disc_overheated`); the `fault_*` files are overlays for damage such as an oil streak or a blade crack.

Group	Files	Examples
`site`	15	foundation, ground, tower sections 1-4, tower door, yaw bearing, earthing, grout
`tower_interior`	18	ladders 1-4, platforms 1-4, lift car and rails, flange bolts, cable loop, ground controller
`nacelle`	10	cover, floor, floor hatch, crane hatch, weather mast, anemometer, aviation light
`bedplate`	1	bedplate
`main_shaft`	10	main shaft, main bearing, rotor lock and pin, shrink disc bolts, grease collector
`gearbox`	11	gearbox, bushings, cover bolts, filter indicator, sight glass
`generator`	10	generator, fast shaft, coupling, brake disc and caliper
`systems`	22	controller and converter cabinets with doors, cooling unit, hydraulic unit, transformer, crane rail, trolley and hook, tool rack, technician
`rotor`	5	hub, blade, blade damage overlays

Placement is not consistent across files (checked on five samples, 30 Oct 2026). All are Z-up, metres, with embedded textures, matching the `private_meshes` convention. But some are local (`tower_section_4` starts at z = 0, `blade` runs from root to tip), some carry the turbine offset in their glTF node transform (`tool_rack`, `brake_disc_ok`), and some have it baked into the vertices (`anemometer_ok` sits about 9 m from its origin). Normalising every part to a sensible local origin, while recording where it sits in the turbine, is part of the work. The turbine's URDF (`windturbine_description`) gives each part's link name and placement.

Tasks

1. **Set up.** Fork `sunava/private_meshes`; run its tests, `scripts/validate_resources.py` and `scripts/build_site.py` locally until they pass unchanged. Clone the GLBs and generator from branch `windturbine_maintenance` and the URDF from `windturbine_description`. Set up a CRAM environment that can import `semantic_digital_twin`.
2. **Learn the target format** from two existing objects: one articulated (`objects/articulated/containers-boxes/139-attached_lid_container/`) and one laboratory device (`objects/laboratory/centrifuge/`): folder layout, `model.urdf`, `metadata.json`, `rights.json`, renders, and the category's `collection.json` entry (id, slug, name, description, size, joints, frames).
3. **Inventory.** List all 102 GLBs with group, matching URDF link, healthy/faulty pairing, whether it is an overlay, file size, and where its placement lives (local, node transform or vertices).
4. **Agree the structure with the maintainer** before converting everything: category name and code (for example `wind-turbine`, `WTB`), the next free ids, how healthy/faulty pairs and `fault_*` overlays are represented (separate objects, or one object with variants), and which parts stay out.
5. **Convert five parts end to end** (suggested: blade, hub, tool rack, controller cabinet with its door, tower section), including renders and catalogue entries, and get them reviewed.
6. **Write the export script** that produces every object folder from the turbine model: normalise each GLB to a local origin, record its placement in the turbine in `metadata.json`, write `model.urdf` (with joints where a part has moving pieces, such as cabinet doors, hatches, rotor lock pin, crane trolley and hook), README, notices and the `collection.json` entry. It belongs next to the existing exporters on `windturbine_maintenance` (or in `windturbine_model`), not in `private_meshes`.
7. **Render previews** (rest pose, and open pose for articulated parts) with an adaptation of `scripts/render_laboratory.py`.
8. **Convert all parts** with the script; add a `semantics.json` per part if the convention from `private_meshes` PR #2 is merged (parts map to `Tower`, `Nacelle`, `Hub`, `RotorBlade`, `Anemometer` on `wind-and-owl`).
9. **Check.** Run the validator, the tests and the site build; load every `model.urdf` with `URDFParser.from_file(...)` in CRAM in one script; look at a sample in CRAMERA and in the web catalogue.

10. **Pull request** to `sunava/private_meshes`, with the inventory, the export command and the check results; iterate on review.
11. **Stretch:** the assembled turbine as `environments/iea_3_4_wind_turbine/` with `environment.urdf`, `scene.json` and the fault scenarios.

Acceptance criteria

- [ ] Every GLB agreed in task 4 has its own folder under the new category, with `model.urdf`, `assets/`, `renders/`, `README.md`, `NOTICE.md`, `LICENSE.txt`, `rights.json` and `metadata.json`
- [ ] All mesh paths are relative; `scripts/validate_resources.py`, the repository's tests and the GitHub Pages build pass
- [ ] Every part appears in the web catalogue with a preview and a working download
- [ ] Every `model.urdf` loads with `URDFParser.from_file(...)` in CRAM, checked by one script
- [ ] Each part's placement in the assembled turbine is recorded, so the turbine can be rebuilt from the objects
- [ ] One command regenerates all folders from the turbine model
- [ ] The pull request is reviewed and merged by the maintainer

Rights and licences

Each object carries its own `rights.json`, `LICENSE.txt` and `NOTICE.md`, as the laboratory devices do. The repository's BSD-3-Clause covers the assets its NOTICE names; new objects state their own terms.

- Author and licence of the turbine meshes: the turbine model's author decides; BSD-3-Clause matches the library.
- Dimensions derive from the IEA-3.4-130-RWT (Apache-2.0): name it, with a link, in every notice.
- Textures come from ambientCG material sets (CC0); `windturbine_description` lists them in `meshes/textures/ATTRIBUTION.txt`.
- The `technician` model's origin is unknown; clear it before including it, or leave it out.

Effort and open questions

Rough estimate for a student assistant at about 10 hours a week, approximately six weeks: one for set-up, format and inventory (tasks 1-3), one for the agreed structure and the first five parts (4-5), two for the export script, renders and all parts (6-8), one for checks and the pull request (9-10), and one buffer. The stretch goal adds about one week.

The repository grows by about 22 MB of GLBs (it is 35 MB today), plus renders.

Open questions, to settle before the student starts:

- Does the maintainer (Vanessa Hassouna) agree to a turbine category, and how should healthy and faulty variants appear?
- Which repository is the turbine model's source: branch `windturbine_maintenance` or `windturbine_model`, and can the student get access to the latter?
- Who is named as author, and under which licence?